arXiv:2502.11089v2 [cs.CL] 27 Feb 2025

Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention

Jingyang Yuan*[1,2], Huazuo Gao[1], Damai Dai[1], Junyu Luo[2], Liang Zhao[1], Zhengyan Zhang[1], Zhenda Xie[1], Y. X. Wei[1], Lean Wang[1], Zhiping Xiao[3], Yuqing Wang[1], Chong Ruan[1], Ming Zhang[2], Wenfeng Liang[1], Wangding Zeng[1]

[1]DeepSeek-AI
[2]Key Laboratory for Multimedia Information Processing, School of Computer Science, Peking University, PKU-Anker LLM Lab
[3]University of Washington
{yuanjy, mzhang_cs}@pku.edu.cn, {zengwangding, wenfeng.liang}@deepseek.com

Abstract

Long-context modeling is crucial for next-generation language models, yet the high computational cost of standard attention mechanisms poses significant computational challenges. Sparse attention offers a promising direction for improving efficiency while maintaining model capabilities. We present NSA, a Natively trainable Sparse Attention mechanism that integrates algorithmic innovations with hardware-aligned optimizations to achieve efficient long-context modeling. NSA employs a dynamic hierarchical sparse strategy, combining coarse-grained token compression with fine-grained token selection to preserve both global context awareness and local precision. Our approach advances sparse attention design with two key innovations: (1) We achieve substantial speedups through arithmetic intensity-balanced algorithm design, with implementation optimizations for modern hardware. (2) We enable end-to-end training, reducing pretraining computation without sacrificing model performance. As shown in Figure 1, experiments show the model pretrained with NSA maintains or exceeds Full Attention models across general benchmarks, long-context tasks, and instruction-based reasoning. Meanwhile, NSA achieves substantial speedups over Full Attention on 64k-length sequences across decoding, forward propagation, and backward propagation, validating its efficiency throughout the model lifecycle.

1. Introduction

The research community increasingly recognizes long-context modeling as a crucial capability for next-generation large language models, driven by diverse real-world applications ranging from in-depth reasoning (DeepSeek-AI, 2025; Zelikman et al., 2022), repository-level code generation (Zhang et al., 2023a; Zhang et al.) and multi-turn autonomous agent systems (Park et al., 2023). Recent breakthroughs, including OpenAI's o-series models, DeepSeek-R1 (DeepSeek-AI, 2025), and Gemini 1.5 Pro (Google et al., 2024), enabling models to process entire codebases, lengthy documents, maintain coherent multi-turn conversations over thousands of tokens, and perform complex reasoning across long-range dependencies. However, the high complexity (Zaheer et al., 2020) of vanilla Attention (Vaswani et al., 2017) mechanisms emerges as a critical

*Contribution during internship at DeepSeek-AI.

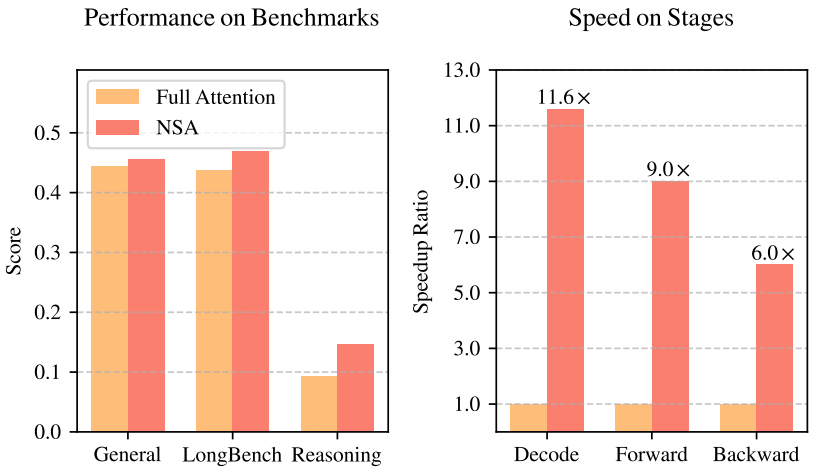

Figure 1 | Comparison of performance and efficiency between Full Attention model and our NSA. Left: Despite being sparse, NSA surpasses Full Attention baseline on average across general benchmarks, long-context tasks, and reasoning evaluation. Right: For 64k-length sequence processing, NSA achieves substantial computational speedup compared to Full Attention in all stages: decoding, forward propagation, and backward propagation.

latency bottleneck as sequence length increases. Theoretical estimates indicate that attention computation with softmax architectures accounts for 70–80% of total latency when decoding 64k-length contexts, underscoring the urgent need for more efficient attention mechanisms.

A natural approach to efficient long-context modeling is to take advantage of the inherent sparsity of softmax attention (Ge et al., 2023; Jiang et al., 2023), where selectively computing critical query-key pairs can significantly reduce computational overhead while preserving performance. Recent advances demonstrate this potential through diverse strategies: KV-cache eviction methods (Li et al., 2024; Zhang et al., 2023b; Zhou et al., 2024), blockwise KV-cache selection methods (Gao et al., 2024; Tang et al., 2024; Xiao et al., 2024a), and sampling, clustering or hashing-based selection methods (Chen et al., 2024b; Desai et al., 2024; Liu et al., 2024). Despite these promising strategies, existing sparse attention methods often fall short in practical deployments. Many approaches fail to achieve speedups comparable to their theoretical gains; moreover, most methods lack effective training-time support to fully exploit the sparsity patterns of attention.

To address these limitations, the deployment of effective sparse attention must tackle two key challenges: (1) ***Hardware-aligned inference speedup***: Converting theoretical computation reductions into actual speed improvements requires hardware-friendly algorithm design during both prefilling and decoding stages to mitigate memory access and hardware scheduling bottlenecks; (2) ***Training-aware algorithm design***: Enabling end-to-end computation with trainable operators to reduce training costs while maintaining model performance. These requirements are crucial for real-world applications to achieve fast long-context inference or training. When considering both aspects, existing methods still exhibit a noticeable gap.

To achieve more effective and efficient sparse attention, we present NSA, a Natively trainable Sparse Attention architecture that integrates hierarchical token modeling. As shown in Figure 2, NSA reduces per-query computation by organizing keys and values into temporal blocks and processing them through three attention paths: compressed coarse-grained tokens, selectively retained fine-grained tokens, and sliding windows for local contextual information. Then

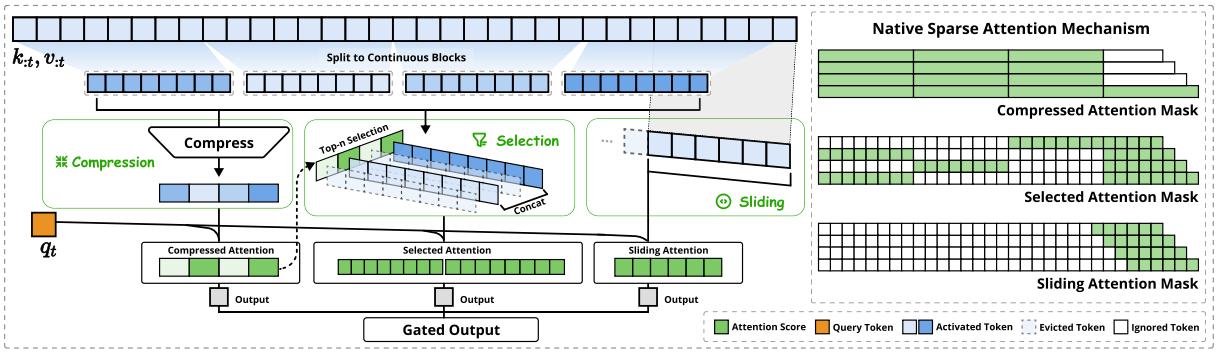

Figure 2 | Overview of NSA's architecture. Left: The framework processes input sequences through three parallel attention branches: For a given query, preceding keys and values are processed into compressed attention for coarse-grained patterns, selected attention for important token blocks, and sliding attention for local context. Right: Visualization of different attention patterns produced by each branch. Green areas indicate regions where attention scores need to be computed, while white areas represent regions that can be skipped.

we implement specialized kernels to maximize its practical efficiency. NSA introduces two core innovations corresponding to the key requirements above: (1) Hardware-aligned system: Optimize blockwise sparse attention for Tensor Core utilization and memory access, ensuring balanced arithmetic intensity. (2) Training-aware design: Enable stable end-to-end training through efficient algorithms and backward operators. This optimization enables NSA to support both efficient deployment and end-to-end training.

We evaluate NSA through comprehensive experiments on real-world language corpora. Pretraining on a 27B-parameter transformer backbone with 260B tokens, we assess NSA's performance across general language evaluations, long-context evaluations, and chain-of-thought reasoning evaluation. We further compare the kernel speed on A100 GPUs with optimized Triton (Tillet et al., 2019) implementations. Experimental results demonstrate that NSA achieves comparable or superior performance to full attention baseline, while outperforming existing sparse attention approaches. Additionally, NSA delivers substantial speedups across decoding, forward, and backward stages compared to Full Attention, with the speedup ratio increasing for longer sequences. These results validate that our hierarchical sparse attention design effectively balances model capability and computational efficiency.

2. Rethinking Sparse Attention Methods

Modern sparse attention methods have made significant strides in reducing the theoretical computational complexity of transformer models. However, most approaches predominantly apply sparsity during inference while retaining a pretrained Full Attention backbone, potentially introducing architectural bias that limits their ability to fully exploit sparse attention's advantages. Before introducing our native sparse architecture, we systematically analyze these limitations through two critical lenses.

2.1. The Illusion of Efficient Inference

Despite achieving sparsity in attention computation, many methods fail to achieve corresponding reductions in inference latency, primarily due to two challenges:

Phase-Restricted Sparsity. Methods such as H2O (Zhang et al., 2023b) apply sparsity during autoregressive decoding while requiring computationally intensive pre-processing (e.g. attention map calculation, index building) during prefilling. In contrast, approaches like MInference (Jiang et al., 2024) focus solely on prefilling sparsity. These methods fail to achieve acceleration across all inference stages, as at least one phase remains computational costs comparable to Full Attention. The phase specialization reduces the speedup ability of these methods in prefilling-dominated workloads like book summarization and code completion, or decoding-dominated workloads like long chain-of-thought (Wei et al., 2022) reasoning.

Incompatibility with Advanced Attention Architecture. Some sparse attention methods fail to adapt to modern decoding efficient architectures like Mulitiple-Query Attention (MQA) (Shazeer, 2019) and Grouped-Query Attention (GQA) (Ainslie et al., 2023), which significantly reduced the memory access bottleneck during decoding by sharing KV across multiple query heads. For instance, in approaches like Quest (Tang et al., 2024), each attention head independently selects its KV-cache subset. Although it demonstrates consistent computation sparsity and memory access sparsity in Multi-Head Attention (MHA) models, it presents a different scenario in models based on architectures like GQA, where the memory access volume of KV-cache corresponds to the union of selections from all query heads within the same GQA group. This architectural characteristic means that while these methods can reduce computation operations, the required KV-cache memory access remains relatively high. This limitation forces a critical choice: while some sparse attention methods reduce computation, their scattered memory access pattern conflicts with efficient memory access design from advanced architectures.

These limitations arise because many existing sparse attention methods focus on KV-cache reduction or theoretical computation reduction, but struggle to achieve significant latency reduction in advanced frameworks or backends. This motivates us to develop algorithms that combine both advanced architectural and hardware-efficient implementation to fully leverage sparsity for improving model efficiency.

2.2. The Myth of Trainable Sparsity

Our pursuit of native trainable sparse attention is motivated by two key insights from analyzing inference-only approaches: (1) ***Performance Degradation***: Applying sparsity post-hoc forces models to deviate from their pretrained optimization trajectory. As demonstrated by Chen et al. (2024b), top 20% attention can only cover 70% of the total attention scores, rendering structures like retrieval heads in pretrained models vulnerable to pruning during inference. (2) ***Training Efficiency Demands***: Efficient handling of long-sequence training is crucial for modern LLM development. This includes both pretraining on longer documents to enhance model capacity, and subsequent adaptation phases such as long-context fine-tuning and reinforcement learning. However, existing sparse attention methods primarily target inference, leaving the computational challenges in training largely unaddressed. This limitation hinders the development of more capable long-context models through efficient training. Additionally, efforts to adapt existing sparse attention for training also expose challenges:

Non-Trainable Components. Discrete operations in methods like ClusterKV (Liu et al., 2024) (includes k-means clustering) and MagicPIG (Chen et al., 2024b) (includes SimHash-based selecting) create discontinuities in the computational graph. These non-trainable components prevent gradient flow through the token selection process, limiting the model's ability to learn optimal sparse patterns.

Inefficient Back-propagation. Some theoretically trainable sparse attention methods suffer

from practical training inefficiencies. Token-granular selection strategy used in approaches like HashAttention (Desai et al., 2024) leads to the need to load a large number of individual tokens from the KV cache during attention computation. This non-contiguous memory access prevents efficient adaptation of fast attention techniques like FlashAttention, which rely on contiguous memory access and blockwise computation to achieve high throughput. As a result, implementations are forced to fall back to low hardware utilization, significantly degrading training efficiency.

2.3. Native Sparsity as an Imperative

These limitations in inference efficiency and training viability motivate our fundamental redesign of sparse attention mechanisms. We propose NSA, a natively sparse attention framework that addresses both computational efficiency and training requirements. In the following sections, we detail the algorithmic design and operator implementation of NSA.

3. Methodology

Our technical approach spans algorithm design and kernel optimization. In the following subsections, we first introduce the background of our methodology. Then we present the overall framework of NSA, followed by its key algorithmic components. Finally, we detail our hardware-optimized kernel design that maximizes practical efficiency.

3.1. Background

Attention Mechanism is widely used in language modeling where each query token $\mathbf{q}_t$ computes relevance scores against all preceding keys $\mathbf{k}_{:t}$ to generate a weighted sum of values $\mathbf{v}_{:t}$. Formally, for an input sequence of length t, the attention operation is defined as:

$$\mathbf{o}_t = \text{Attn}\left(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}\right) \tag{1}$$

where Attn denotes the attention function:

$$\text{Attn}\left(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}\right) = \sum_{i=1}^{t} \frac{\alpha_{t,i}\mathbf{v}_i}{\sum_{j=1}^{t}\alpha_{t,j}}, \quad \alpha_{t,i} = e^{\frac{\mathbf{q}_t^\top \mathbf{k}_i}{\sqrt{d_k}}}. \tag{2}$$

Here, $\alpha_{t,i}$ represents the attention weight between $\mathbf{q}_t$ and $\mathbf{k}_i$, and d_k is the feature dimension of keys. As sequence length increases, attention computation becomes increasingly dominant in the overall computational cost, presenting significant challenges for long-context processing.

Arithmetic Intensity is the ratio of compute operations to memory accesses. It intrinsically shapes algorithm optimization on hardware. Each GPU has a critical arithmetic intensity determined by its peak compute capability and memory bandwidth, calculated as the ratio of these two hardware limits. For computation tasks, arithmetic intensity above this critical threshold becomes compute-bound (limited by GPU FLOPS), while below it becomes memory-bound (limited by memory bandwidth).

Specifically for causal self-attention mechanism, during training and prefilling phases, batched matrix multiplications and attention computations exhibit high arithmetic intensity, making these stages compute-bound on modern accelerators. In contrast, auto-regressive decoding becomes memory-bandwidth constrained because it generates one token per forward pass while requiring loading the entire key-value cache, resulting in low arithmetic intensity.

This leads to different optimization goals — reducing computation cost during training and prefilling, while reducing memory access during decoding.

3.2. Overall Framework

To leverage the potential of attention with natural sparse pattern, we propose replacing the original key-value pairs $\mathbf{k}_{:t}, \mathbf{v}_{:t}$ in Equation (1) with a more compact and information-dense set of representation key-value pairs $\tilde{K}_t, \tilde{V}_t$ given each query $\mathbf{q}_t$. Specifically, we formally define the optimized attention output as follows:

$$\tilde{K}_t = f_K(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}), \quad \tilde{V}_t = f_V(\mathbf{q}_t, \mathbf{k}_{:t}, \mathbf{v}_{:t}) \tag{3}$$

$$\mathbf{o}_t^* = \text{Attn}\left(\mathbf{q}_t, \tilde{K}_t, \tilde{V}_t\right) \tag{4}$$

where $\tilde{K}_t, \tilde{V}_t$ are dynamically constructed based on the current query $\mathbf{q}_t$ and the contextual memory $\mathbf{k}_{:t}, \mathbf{v}_{:t}$. We can design various mapping strategies to get different categories of $\tilde{K}_t^c, \tilde{V}_t^c$, and combine them as follows:

$$\mathbf{o}_t^* = \sum_{c \in C} g_t^c \cdot \text{Attn}(\mathbf{q}_t, \tilde{K}_t^c, \tilde{V}_t^c). \tag{5}$$

As illustrated in Figure 2, NSA have three mapping strategies $C = \{\text{cmp}, \text{slc}, \text{win}\}$, representing compression, selection, and sliding window for keys and values. $g_t^c \in [0, 1]$ is the gate score for corresponding strategy c, derived from input features via an MLP and sigmoid activation. Let N_t denote the total number of remapped keys/values:

$$N_t = \sum_{c \in C} \text{size}[\tilde{K}_t^c]. \tag{6}$$

We maintain a high sparsity ratio by ensuring$N_t \ll t$.

3.3. Algorithm Design

In this subsection, we introduce the design of our remapping strategies f_K and f_V: token compression, token selection, and sliding window.

3.3.1. Token Compression

By aggregating sequential blocks of keys or values into block-level representations, we obtain compressed keys and values that capture the information of the entire block. Formally, the compressed key representation is defined as:

$$\tilde{K}_t^{\text{cmp}} = f_K^{\text{cmp}}(\mathbf{k}_{:t}) = \left\{ \varphi(\mathbf{k}_{id+1:id+l}) \middle| 0 \leqslant i \leqslant \left\lfloor \frac{t-l}{d} \right\rfloor \right\} \tag{7}$$

where l is the block length, d is the sliding stride between adjacent blocks, and φ is a learnable MLP with intra-block position encoding to map keys in a block to a single compressed key. $\tilde{K}_t^{\text{cmp}} \in \mathbb{R}^{d_k \times \lfloor \frac{t-l}{d} \rfloor}$ is tensor composed by compresion keys. Usually, we adopt $d < l$ to mitigate information fragmentation. An analogous formulation holds for the compressed value representation $\tilde{V}_t^{\text{cmp}}$. Compressed representations capture coarser-grained higher-level semantic information and reduce computational burden of attention.

3.3.2. Token Selection

Using only compressed keys, values might lose important fine-grained information, motivating us to selectively preserve individual keys, values. Below we describe our efficient token selection mechanism that identifies and preserves the most relevant tokens with low computational overhead.

Blockwise Selection. Our selection strategy processes key and value sequences in spacial continuous blocks, motivated by two key factors: hardware efficiency considerations and inherent distribution patterns of attention scores. ***Blockwise selection is crucial to achieve efficient computation on modern GPUs.*** That is because modern GPU architectures exhibit significantly higher throughput for continuous block accesses compared to random index-based reads. Also, blockwise computation enables optimal utilization of Tensor Cores. This architectural characteristic has established blockwise memory access and computation as a fundamental principle in high-performance attention implementations, as exemplified by FlashAttention's block-based design. ***Blockwise selection follows the inherent distribution patterns of attention scores.*** Prior works (Jiang et al., 2024) have shown that attention scores often exhibit spatial continuity, suggesting that neighboring keys tend to share similar importance levels. Our visualization in Section 6.2 also shows this spatial continuous pattern.

To implement blockwise selection, we first divide key, value sequences into selection blocks. To identify the most important blocks for attention computation, we need to assign importance scores to each block. Below we present our method for computing these block-level importance scores.

Importance Score Computation. Computing block importance scores could introduce significant overhead. Fortunately, the attention computation of compression tokens produces intermediate attention scores that we can leverage to induce selection block importance scores, formulated as:

$$\mathbf{p}_t^{\text{cmp}} = \text{Softmax}\left(\mathbf{q}_t^T \tilde{K}_t^{\text{cmp}}\right), \tag{8}$$

where $\mathbf{p}_t^{\text{cmp}} \in \mathbb{R}^{\lfloor \frac{t-l}{d} \rfloor + 1}$ is the attention scores between q_t and compression keys $\tilde{K}_t^{\text{cmp}}$. Let l' denote the selection block size. When compression blocks and selection blocks share the same blocking scheme, i.e., $l' = l = d$, we can directly obtain the selection block importance scores $\mathbf{p}_t^{\text{slc}}$ by $\mathbf{p}_t^{\text{slc}} = \mathbf{p}_t^{\text{cmp}}$ straightforwardly. For cases where the blocking schemes differ, we derive the importance scores for selection blocks according to their spatial relationship. Given $l \leqslant l'$, $d \mid l$ and $d \mid l'$, we have:

$$\mathbf{p}_t^{\text{slc}}[j] = \sum_{m=0}^{\frac{l'}{d}-1} \sum_{n=0}^{\frac{l}{d}-1} \mathbf{p}_t^{\text{cmp}} \left[\frac{l'}{d} j - m - n\right], \tag{9}$$

where$[\cdot]$ denotes the indexing operator for accessing vector element. For models employing GQA or MQA where key-value caches are shared across query heads, consistent block selection across these heads has to be ensured to minimize KV cache loading during decoding. The shared importance scores across heads in a group are formally defined as:

$$\mathbf{p}_t^{\text{slc}'} = \sum_{h=1}^{H} \mathbf{p}_t^{\text{slc},(h)}, \tag{10}$$

where (h) in the superscript denotes the head index, and H is the number of query heads in each group. This aggregation ensures consistent block selection across heads within the same group.

Top-n Block Selection. After obtaining the selection block importance scores, We retain tokens within the top-n sparse blocks ranked by block importance scores, formulated as:

$$\mathcal{I}_t = \{i \mid \text{rank}(\mathbf{p}_t^{\text{slc}'}[i]) \leqslant n\} \tag{11}$$

$$\tilde{K}_t^{\text{slc}} = \text{Cat}\left[\{\mathbf{k}_{il'+1:(i+1)l'} | i \in \mathcal{I}_t\}\right], \tag{12}$$

where rank$(\cdot)$ denotes the ranking position in descending order, with rank = 1 corresponding to the highest score, $\mathcal{I}_t$ is the set of selected blocks' indices, Cat denotes the concatenation operation. $\tilde{K}_t^{\text{slc}} \in \mathbb{R}^{d_k \times nl'}$ is tensor composed by compresion keys. An analogous formulation applies to the fine-grained value $\tilde{V}_t^{\text{slc}}$. The selected keys and values then participate in the attention computation with $\mathbf{q}_t$ as defined in Equation (5).

3.3.3. Sliding Window

In attention mechanisms, local patterns typically adapt faster and can dominate the learning process, potentially preventing the model from effectively learning from compression and selection tokens. To address this issue, we introduce a dedicated sliding window branch that explicitly handles local context, allowing other branches (compression and selection) to focus on learning their respective features without being shortcutted by local patterns. Specifically, we maintain recent tokens $\tilde{K}_t^{\text{win}} = \mathbf{k}_{t-w:t}, \tilde{V}_t^{\text{win}} = \mathbf{v}_{t-w:t}$ in a window w, and isolate attention computations of different information sources (compression tokens, and selected tokens, sliding window) into separate branches. These branch outputs are then aggregated through a learned gating mechanism. To further prevent shortcut learning across attention branches with marginal computational overhead, we provide independent keys and values for three branches. This architectural design enables stable learning by preventing gradient interference between local and long-range pattern recognition, while introducing minimal overhead.

After obtaining all three categories of keys and values ($\tilde{K}_t^{\text{cmp}}, \tilde{V}_t^{\text{cmp}}$; $\tilde{K}_t^{\text{slc}}, \tilde{V}_t^{\text{slc}}$; and $\tilde{K}_t^{\text{win}}, \tilde{V}_t^{\text{win}}$), we compute the final attention output following Equation (5). Together with the compression, selection, and sliding window mechanisms described above, this forms the complete algorithmic framework of NSA.

3.4. Kernel Design

To achieve FlashAttention-level speedup during the training and prefilling, we implement hardware-aligned sparse attention kernels upon Triton. Given MHA is memory-intensive and inefficient for decoding, we focus on architectures with shared KV caches like GQA and MQA following the current state-of-the-art LLMs. While compression and sliding window attention computations are readily compatible with existing FlashAttention-2 kernels, we introduce the specialized kernel design for sparse selection attention. If we were to follow FlashAttention's strategy of loading temporally continuous query blocks into SRAM, it would result in inefficient memory access since queries within a block may require disjoint KV blocks. To address this, our key optimization lies in a different query grouping strategy: for each position on the query sequence, we load all query heads within a GQA group (they share the same sparse KV blocks) into SRAM. Figure 3 illustrates our forward pass implementation. The proposed kernel architecture is characterized by the following key features:

1. **Group-Centric Data Loading**. For each inner loop, load all heads' queries $Q \in \mathbb{R}^{[h,d_k]}$ in the group at position t and their shared sparse key/value block indices $\mathcal{I}_t$.

2. **Shared KV Fetching**. In the inner loop, Sequentially load continuous key/value blocks indexed by $\mathcal{I}_t$ into SRAM as $K \in \mathbb{R}^{[B_k, d_k]}, V \in \mathbb{R}^{[B_k, d_v]}$ to minimize memory loading, where B_k is the kernel block size satisfying $B_k | l'$.
3. **Outer Loop on Grid**. Since the inner-loop length (proportional to the selected block count n) remains nearly identical for different query blocks, we put query/output loops in Triton's grid scheduler to simplify and optimize the kernel.

This design achieves near-optimal arithmetic intensity by (1) eliminating redundant KV transfers through group-wise sharing, and (2) balancing compute workloads across GPU streaming multiprocessors.

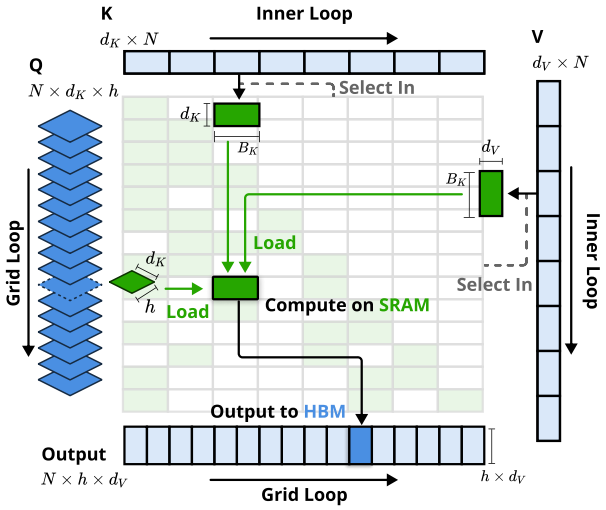

Figure 3 | Kernel design for NSA. The kernel loads queries by GQA groups (Grid Loop), fetches corresponding sparse KV blocks (Inner Loop), and performs attention computation on SRAM. Green blocks indicate data on SRAM, while blue indicates data on HBM.

4. Experiments

We evaluate NSA through three lenses: (1) general benchmarks performance, (2) long-context benchmarks performance, and (3) chain-of-thought reasoning performance, comparing against Full Attention baseline and state-of-the-art sparse attention methods. We defer the efficiency analysis of our sparse computation paradigm to Section 5, where we provide detailed discussions on training and inference speed.

4.1. Pretraining Setup

Following the common practice in state-of-the-art LLMs, our experiments adopt a backbone combining Grouped-Query Attention (GQA) and Mixture-of-Experts (MoE), featuring 27B total parameters with 3B active parameters. The model consists of 30 layers with a hidden dimension of 2560. For GQA, we set the number of groups to 4, with a total of 64 attention heads. For each head, the hidden dimensions of the query, key, and value are configured as $d_q = d_k = 192$ and $d_v = 128$, respectively. For MoE, we utilize the DeepSeekMoE (Dai et al., 2024; DeepSeek-AI, 2024) structure, with 72 routed experts and 2 shared experts, and set the top-k experts to 6. To ensure training stability, the MoE in the first layer is replaced by an MLP in the form of SwiGLU.

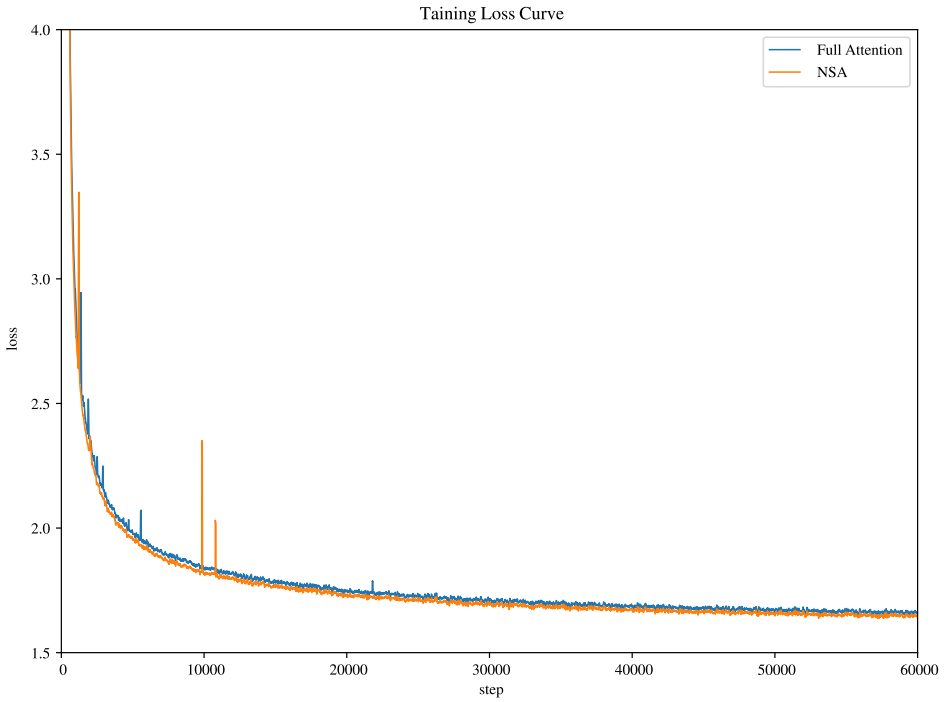

Figure 4 | Pretraining loss comparison between Full Attention and our NSA on 27B-parameter model. Both models exhibit stable convergence, with NSA achieving lower loss values.

Model	MMLU Acc. 5-shot	MMLU-PRO Acc. 5-shot	CMMLU Acc. 5-shot	BBH Acc. 3-shot	GSM8K Acc. 8-shot	MATH Acc. 4-shot	DROP F1 1-shot	MBPP Pass@1 3-shot	HumanEval Pass@1 0-shot	Avg.
Full Attn	**0.567**	0.279	0.576	0.497	0.486	0.263	0.503	**0.482**	0.335	0.443
NSA	0.565	**0.286**	**0.587**	**0.521**	**0.520**	**0.264**	**0.545**	0.466	**0.348**	**0.456**

Table 1 | Pretraining performance comparison between the full attention baseline and NSA on general benchmarks, across knowledge (MMLU, MMLU-PRO, CMMLU), reasoning (BBH, GSM8K, MATH, DROP), and coding (MBPP, HumanEval) tasks. NSA achieves superior average performance on most benchmarks despite high sparsity.

The proposed architecture achieves an effective trade-off between computation cost and model performance. For NSA, we set compression block size $l = 32$, sliding stride $d = 16$, selected block size $l' = 64$, selected block count $n = 16$ (including fixed activating the 1 initial block and 2 local blocks), and sliding window size $w = 512$. Both Full Attention and sparse attention models are pretrained on 270B tokens of 8k-length texts, followed by continued training and supervised fine-tuning on 32k-length texts with YaRN (Peng et al., 2024) to achieve long-context adaptation. Both models are trained to full convergence to ensure fair comparison. As shown in Figure 4, the pretraining loss curve of our NSA and Full Attention baseline demonstrates stable and smooth decline, with NSA consistently outperforming the Full Attention model.

4.2. Baselines Methods

In addition to comparing with Full Attention, we evaluate several state-of-the-art inference-stage sparse attention methods: H2O (Zhang et al., 2023b), infLLM (Xiao et al., 2024a), Quest (Tang et al., 2024), and Exact-Top, which first computes full attention score and select the top-n scores keys corresponding to each query and then calculates attention on these positions. These

Model	SQA			MQA				Synthetic		Code	Avg.
	MFQA-en	MFQA-zh	Qasper	HPQ	2Wiki	GovRpt	Dur	PassR-en	PassR-zh	LCC	
H2O	0.428	0.429	0.308	0.112	0.101	0.231	0.208	0.704	0.421	0.092	0.303
InfLLM	0.474	0.517	0.356	0.306	0.250	0.277	0.257	0.766	0.486	0.143	0.383
Quest	0.495	0.561	0.365	0.295	0.245	0.293	0.257	0.792	0.478	0.135	0.392
Exact-Top	0.502	0.605	0.397	0.321	0.288	0.316	0.291	0.810	0.548	0.156	0.423
Full Attn	**0.512**	0.623	0.409	0.350	0.305	**0.324**	0.294	0.830	**0.560**	0.163	0.437
NSA	0.503	**0.624**	**0.432**	**0.437**	**0.356**	0.307	**0.341**	**0.905**	0.550	**0.232**	**0.469**

Table 2 | Performance comparison between our NSA and baselines on LongBench, including subsets in single document QA, multi-document QA, synthetic and code task categories. NSA outperformed most of the baselines including Full Attention.

methods span diverse sparse attention paradigms, including KV-cache eviction, query-aware selection, and exact top-n sparse selection.

For general evaluation, where most samples have lengths within the local context window of sparse attention baselines, these methods are effectively equivalent to Full Attention. Therefore, we present only the comparison results between NSA and Full Attention baseline in this setting. In the long-context evaluation, we conduct comparisons across all baseline methods, with the sparsity of all sparse attention methods set to the same to ensure a fair comparison. For chain-of-thought reasoning evaluation, which requires long-text supervised fine-tuning, we limit our comparison to Full Attention, as sparse attention baselines do not support training.

4.3. Performance Comparison

General Evaluation. We evaluated the pretrained NSA and Full Attention baseline, on a comprehensive suite of benchmarks spanning knowledge, reasoning, and coding capabilities, including MMLU (Hendrycks et al., 2020), MMLU-PRO (Wang et al., 2024), CMMLU (Li et al., 2023), BBH (Suzgun et al., 2022), GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2020), DROP (Dua et al., 2019), MBPP (Austin et al., 2021), and HumanEval (Chen et al., 2021). The results are shown in Table 1. Despite its sparsity, NSA achieves superior overall performance, outperforming all baselines including Full Attention on 7 out of 9 metrics. This indicates that although NSA may not fully leverage its efficiency advantages on shorter sequences, it shows strong performance. Notably, NSA demonstrates significant gains in reasoning-related benchmarks (DROP: +0.042, GSM8K: +0.034), suggesting that our pretraining helps models to develop specialized attention mechanisms. This sparse attention pretraining mechanism forces model to focus on the most important information, potentially enhancing performance by filtering out noise from irrelevant attention pathways. The consistent performance across diverse evaluations also validates NSA's robustness as a general-purpose architecture.

Long-Context Evaluation. As shown in Figure 5, NSA achieves perfect retrieval accuracy across all positions in 64k-context needle-in-a-haystack (Kamradt, 2023) test. This performance stems from our hierarchical sparse attention design, which combines compression tokens for efficient global context scanning, and selection tokens for precise local information retrieval. The coarse-grained compression identifies relevant context blocks at low computational cost, while the token-level attention on selected tokens ensures the preservation of critical fine-grained information. This design enables NSA to maintain both global awareness and local precision.

We also evaluate NSA on LongBench (Bai et al., 2023) against state-of-the-art sparse attention

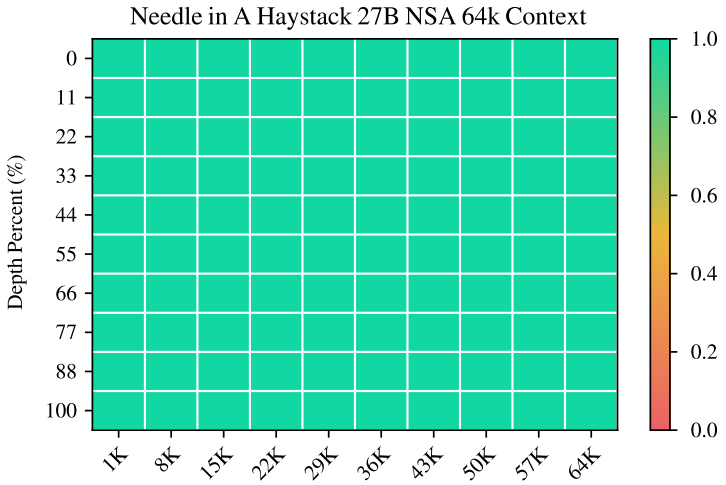

Figure 5 | Needle-in-a-Haystack retrieval accuracy across context positions with 64k context length. NSA achieves perfect accuracy through its hierarchical sparse attention design.

methods and Full Attention baseline. To ensure consistent sparsity, we set the token activated by each query in all sparse attention baselines to 2560 tokens, which corresponds to the average number of tokens activated in NSA when handling 32k sequence lengths. Following StreamLLM (Xiao et al., 2023), this token budget includes the leading 128 tokens and 512 local tokens. We exclude certain subsets from LongBench due to their low scores across all models, which may not provide meaningful comparisons. As shown in Table 2, NSA achieves the highest average score 0.469, outperforming all baselines (+0.032 over Full Attention and +0.046 over Exact-Top). This improvement arises from two key innovations: (1) our native sparse attention design, which enables end-to-end optimization of sparse patterns during pretraining, facilitates synchronized adaptation between the sparse attention module and other model components; and (2) the hierarchical sparse attention mechanism achieves a balance between local and global information processing.

Notably, NSA demonstrates exceptional performance on tasks requiring complex reasoning over long contexts, achieving +0.087 and +0.051 improvements over Full Attention on multi-hop QA tasks (HPQ and 2Wiki), exceeding the performance of baselines on code understanding (LCC: +0.069), and outperforming other methods on passage retrieval (PassR-en: +0.075). These results validate NSA's capability to handle diverse long-context challenges, with its natively pretrained sparse attention providing additional benefits in learning task-optimal patterns.

Chain-of-Thought Reasoning Evaluation. To evaluate NSA's compatibility with advanced downstream training paradigms, we investigate its capacity to acquire chain-of-thought mathematical reasoning abilities via post-training. Given the limited effectiveness of reinforcement learning on smaller-scale models, we employ knowledge distillation from DeepSeek-R1, conducting supervised fine-tuning (SFT) with 10B tokens of 32k-length mathematical reasoning traces. This produces two comparable models: Full Attention-R (Full Attention baseline) and NSA-R (our sparse variant). We assess both models on the challenging American Invitational Mathematics Examination (AIME 24) benchmark. We use a sampling temperature of 0.7 and a top-p value of 0.95 to generate 16 responses for each question and obtain the average score. To validate the impact of reasoning depth, we conduct experiments with two generation context limits: 8k and 16k tokens, measuring whether extended reasoning chains improve accuracy. Example comparisons of model predictions are provided in Appendix A.

Generation Token Limit	8192	16384
Full Attention-R	0.046	0.092
NSA-R	**0.121**	**0.146**

Table 3 | AIME Instruction-based Evaluating after supervised fine-tuning. Our NSA-R demonstrates better performance than Full Attention-R at both 8k and 16k sequence lengths

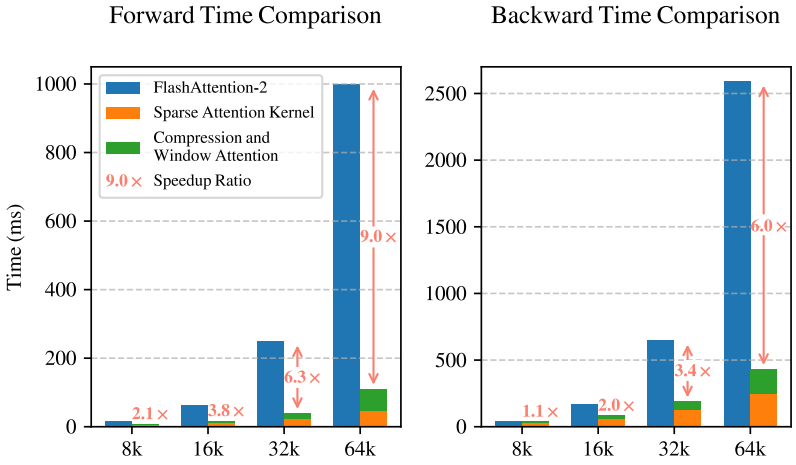

Figure 6 | Comparison of Triton-based NSA kernel with Triton-based FlashAttention-2 kernel. Our implementation significantly reduces latency across all context lengths, with the improvement becoming more pronounced as input length increases.

As shown in Table 3, NSA-R achieves significantly higher accuracy than Full Attention-R under the 8k context setting (+0.075), with this advantage persisting at 16k contexts (+0.054). These results validate two key benefits of native sparse attention: (1) The pretrained sparse attention patterns enable efficient capture of long-range logical dependencies critical for complex mathematical derivations; (2) Our architecture's hardware-aligned design maintains sufficient context density to support growing reasoning depth without catastrophic forgetting. The consistent outperformance across context lengths confirms sparse attention's viability for advanced reasoning tasks when natively integrated into the training pipeline.

5. Efficiency Analysis

We evaluate the computational efficiency of NSA against Full Attention on an 8-GPU A100 system. In efficiency analysis, we also configure the model with GQA group $g = 4$, heads per group $h = 16$, query/key dimension $d_k = 192$, and value dimension $d_v = 128$. Following the same settings in Section 4, we set NSA compression block size $l = 32$, sliding stride $d = 16$, selected block size $l' = 64$, selected block count $n = 16$, and sliding window size $w = 512$.

5.1. Training Speed

We compare the Triton-based implementations of our NSA attention and Full Attention with Triton-based FlashAttention-2 to ensure fair speed comparison across the same backend. As shown in Figure 6, our NSA achieves progressively greater speedups as context length increases, up to 9.0× forward and 6.0× backward speedup at 64k context-length. Notably, the speed advantage becomes more pronounced with longer sequences. This speedup stems from our

Context Length	8192	16384	32768	65536
Full Attention	8192	16384	32768	65536
NSA	2048	2560	3584	5632
Expected Speedup	**4**×	**6.4**×	**9.1**×	**11.6**×

Table 4 | Memory access volume (in equivalent number of tokens) per attention operation during decoding. Due to the low arithmetic intensity and memory-bound nature of decoding, the expected speedup is approximately linear with the volume of memory access.

hardware-aligned algorithm design to maximize the efficiency of sparse attention architecture: (1) The Blockwise memory access pattern maximizes Tensor Core utilization through coalesced loads, (2) The delicate loop scheduling in the kernel eliminates redundant KV transfers.

5.2. Decoding Speed

The decoding speed of Attention is primarily determined by the memory access bottleneck, which is closely tied to the amount of KV cache loading. In each decoding step, Our NSA just needs to load at most $\lfloor \frac{s-l}{d} \rfloor$ compression tokens, nl' selected tokens, and w neighbor tokens, where s is the cached sequence length. As shown in Table 4, our method exhibits a significant reduction in latency as the decoding length increases, achieving up to 11.6× speedup at 64k context-length. This advantage in memory access efficiency also amplifies with longer sequences.

6. Discussion

In this section, we reflect on the development process of NSA and discuss key insights gained from our exploration of different sparse attention strategies. While our approach demonstrates promising results, understanding the challenges encountered with alternative strategies and analyzing attention patterns provides valuable context for future research directions. We first examine challenges with alternative token selection strategies that motivated our design choices, followed by visualizations that offer insights into attention distribution patterns.

6.1. Challenges with Alternative Token Selection Strategies

Before designing NSA, we explored adapting existing sparse attention methods to the training stage. However, these attempts encountered various challenges, prompting us to design a different sparse attention architecture:

Key-Clustering Based Strategies. We examined clustering-based strategies like ClusterKV (Liu et al., 2024). These methods store Keys and Values from the same cluster in contiguous memory regions. While theoretically feasible for training and inference, they face three significant challenges: (1) Non-trivial computational overhead introduced by dynamic clustering mechanisms; (2) Operator optimization difficulties exacerbated by inter-cluster imbalances, especially in Mixture-of-Experts (MoE) systems, where skewed Expert Parallelism (EP) group execution times lead to persistent load imbalances; (3) Implementation constraints arising from the need for mandatory periodic reclustering and chunk-sequential training protocols. These combined factors create substantial bottlenecks, significantly limiting their effectiveness for real-world deployment.

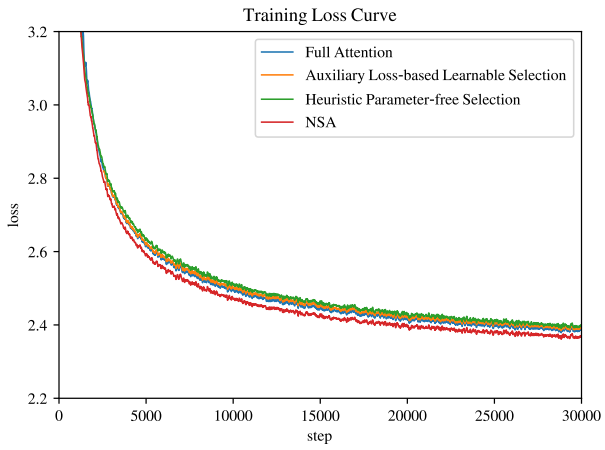

Figure 7 | Compare training loss on a 3B-parameter model with Full Attention and different token selection strategies and. Our NSA achieves better performance.

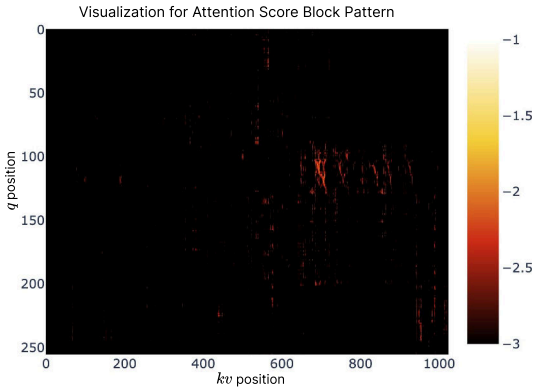

Figure 8 | Visualization of Attention Map on a Full Attention transformer. Light-colored regions indicate higher attention values. As shown in the figure, attention scores exhibit blockwise clustering distribution.

Other Blockwise Selection Strategies. We also considered blockwise key, value selection strategies different from NSA, such as Quest (Tang et al., 2024) and InfLLM (Xiao et al., 2024a). These methods rely on computing an importance score for each KV block and selecting the top-n blocks based on their similarity with q_t. However, existing methods face two critical issues: (1) Since the selection operation is non-differentiable, importance score computation based on neural networks relies on auxiliary loss, which increases operator overhead and often degrades model performance; (2) Heuristic parameter-free importance score computation strategy suffer from low recall rates, leading to suboptimal performance. We evaluate both approaches on a 3B-parameter model with similar architecture and compare their loss curve with NSA and Full Attention. For the auxiliary loss-based selection method, we introduce additional queries for each token and representative keys for each block to estimate the block importance scores. We compute block-level supervision signals by mean-pooling attention scores within each key block, and use KL divergence to supervise block importance prediction. We maintain individual query granularity instead of block-averaged queries to accommodate efficient decoding. This auxiliary loss-based importance estimation shares conceptual similarity with SeerAttention (Gao et al., 2024). For the heuristic parameter-free selection method, following the strategy of Quest, we implement direct selection using the product between queries and coordinate-wise min-max of the key chunks, without introducing additional parameters. We also explore a cold-start training approach where Full Attention is applied for the initial 1000 steps before transitioning to the heuristic blockwise selection. As shown in Figure 7, both methods exhibited inferior loss.

6.2. Visualization

To explore potential patterns in transformer attention distributions and seek inspiration for our design, we visualize the attention map from our pretrained 27B Full Attention model in Figure 8. The visualization reveals interesting patterns where attention scores tend to exhibit blockwise clustering characteristics, with nearby keys often showing similar attention scores. This observation inspired our design of NSA, suggesting that selecting key blocks based on spatial continuity might be a promising approach. The blockwise clustering phenomenon indicates that tokens adjacent in the sequence may share certain semantic relationships with query tokens, though the exact nature of these relationships requires further investigation. This

observation motivated us to explore a sparse attention mechanism that operates on continuous token blocks rather than individual tokens, aiming to enhance computational efficiency and preserve high-attention patterns.

7. Related Works

We review existing approaches that improve the efficiency of attention computation through sparse attention. These methods can be broadly categorized into three groups based on their core strategies: (1) fixed sparse pattern, (2) dynamic token pruning, and (3) query-aware selection. We introduce several representative works from each category.

7.1. Fixed Sparse Pattern

SlidingWindow is a commonly used approach that allows the query to compute attention only within a fixed window. StreamingLLM (Xiao et al., 2023) combines attention sinks with local windows to process continuous text streams. MoA (Fu et al., 2024a) and DuoAttention (Xiao et al., 2024b) also adopt similar local and sink information for long sequence modeling. Longformer (Beltagy et al., 2020) interleaves local windowed attention with global tokens to process long sequences. Compared with them, our NSA does not rely on pre-defined sparse patterns, but learns the patterns automatically, unlocking the potential to utilize full context.

7.2. Dynamic Token Pruning

Some methods are designed to reduce memory and computation costs during inference through dynamic KV-cache pruning. H2O (Zhang et al., 2023b), BUZZ (Zhao et al., 2024), and SepLLM (Chen et al., 2024a) implement adaptive approaches to reduce KV-cache memory usage during decoding. These methods dynamically evict tokens deemed less important for future predictions. FastGen Ge et al. (2023) and HeadKV (Fu et al., 2024b) optimize computation by assigning different strategies to individual attention heads. SnapKV (Li et al., 2024) introduces a token pruning strategy that reduces the KV-cache by selectively retaining only the most crucial features, enabling efficient memory usage. Unlike these inference-focused approaches, our NSA incorporates sparsity natively during the training phase.

7.3. Query-Aware Selection

Other works focus on query-dependent token selection methods to reduce attention computation while preserving attention quality. Quest (Tang et al., 2024) employs a blockwise selection strategy where each chunk's importance is estimated by product between query and coordinate-wise min-max of the key chunks. InfLLM (Xiao et al., 2024a) combines fixed patterns with retrieval by maintaining attention sinks, local context, and retrievable chunks. This method selects representative keys from each chunk to estimate chunk importance. HashAttention (Desai et al., 2024) formulates pivotal token identification as a recommendation problem by mapping queries and keys to Hamming space using learned functions. ClusterKV (Liu et al., 2024) achieves sparsity by firstly clustering keys and then selecting the most relevant clusters for attention computation based on query-cluster similarity. MInference (Jiang et al., 2024) and TokenSelect (Wu et al., 2024) select KV pairs for computation based on token-level importance scoring. SeerAttention (Gao et al., 2024) separates queries and keys into spatial blocks and performs blockwise selection for efficient computation. Compared to these methods, our NSA

achieves hardware-aligned sparse attention computation throughout the full model lifecycle, including training, prefilling, and decoding.

8. Conclusion

We present NSA, a hardware-aligned sparse attention architecture for efficient long-context modeling. By integrating hierarchical token compression with blockwise token selection within a trainable architecture, our architecture achieves accelerated training and inference while maintaining Full Attention performance. NSA advances the state-of-the-art by demonstrating general benchmark performance matches full-attention baselines, exceeding modeling capability in long-context evaluations, and enhanced reasoning ability, all accompanied by measurable reductions in computational latency and achieving significant speedup.

References

J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245, 2023.

J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021.

Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. arXiv preprint arXiv:2308.14508, 2023.

I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150, 2020.

G. Chen, H. Shi, J. Li, Y. Gao, X. Ren, Y. Chen, X. Jiang, Z. Li, W. Liu, and C. Huang. Sepllm: Accelerate large language models by compressing one segment into one separator. arXiv preprint arXiv:2412.12094, 2024a.

M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

Z. Chen, R. Sadhukhan, Z. Ye, Y. Zhou, J. Zhang, N. Nolte, Y. Tian, M. Douze, L. Bottou, Z. Jia, et al. Magicpig: Lsh sampling for efficient llm generation. arXiv preprint arXiv:2410.16179, 2024b.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168, 2021.

D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. arXiv preprint arXiv:2401.06066, 2024.

DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. 2024. URL https://arxiv.org/abs/2405.04434.

DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

A. Desai, S. Yang, A. Cuadron, A. Klimovic, M. Zaharia, J. E. Gonzalez, and I. Stoica. Hashattention: Semantic sparsity for faster inference. arXiv preprint arXiv:2412.14468, 2024.

D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, and M. Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. arXiv preprint arXiv:1903.00161, 2019.

T. Fu, H. Huang, X. Ning, G. Zhang, B. Chen, T. Wu, H. Wang, Z. Huang, S. Li, S. Yan, et al. Moa: Mixture of sparse attention for automatic large language model compression. arXiv preprint arXiv:2406.14909, 2024a.

Y. Fu, Z. Cai, A. Asi, W. Xiong, Y. Dong, and W. Xiao. Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning. arXiv preprint arXiv:2410.19258, 2024b.

Y. Gao, Z. Zeng, D. Du, S. Cao, H. K.-H. So, T. Cao, F. Yang, and M. Yang. Seerattention: Learning intrinsic sparse attention in your llms. arXiv preprint arXiv:2410.13276, 2024.

S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao. Model tells you what to discard: Adaptive kv cache compression for llms. arXiv preprint arXiv:2310.01801, 2023.

G. T. Google, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530, 2024.

D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300, 2020.

H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. arXiv preprint arXiv:2310.05736, 2023.

H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, et al. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. arXiv preprint arXiv:2407.02490, 2024.

G. Kamradt. LLMTest NeedleInAHaystack. GitHub repository, 2023. URL https://github.com/gkamradt/LLMTest_NeedleInAHaystack. Accessed: [Insert Access Date Here].

H. Li, Y. Zhang, F. Koto, Y. Yang, H. Zhao, Y. Gong, N. Duan, and T. Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese. arXiv preprint arXiv:2306.09212, 2023.

Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen. Snapkv: Llm knows what you are looking for before generation. arXiv preprint arXiv:2404.14469, 2024.

G. Liu, C. Li, J. Zhao, C. Zhang, and M. Guo. Clusterkv: Manipulating llm kv cache in semantic space for recallable compression. arXiv preprint arXiv:2412.03213, 2024.

J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In S. Follmer, J. Han, J. Steimle, and N. H. Riche, editors, Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023– 1 November 2023, pages 2:1–2:22. ACM, 2023.

B. Peng, J. Quesnelle, H. Fan, and E. Shippole. Yarn: Efficient context window extension of large language models. In ICLR. OpenReview.net, 2024.

N. Shazeer. Fast transformer decoding: One write-head is all you need. CoRR, abs/1911.02150, 2019.

M. Suzgun, N. Scales, N. Schärli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Chi, D. Zhou, et al. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint arXiv:2210.09261, 2022.

J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han. Quest: Query-aware sparsity for efficient long-context llm inference. arXiv preprint arXiv:2406.10774, 2024.

P. Tillet, H.-T. Kung, and D. Cox. Triton: an intermediate language and compiler for tiled neural network computations. In Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages, pages 10–19, 2019.

A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin. Attention is all you need. Advances in Neural Information Processing Systems, 2017.

Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang, et al. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. arXiv preprint arXiv:2406.01574, 2024.

J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. Advances in neural information processing systems, 35:24824–24837, 2022.

W. Wu, Z. Pan, C. Wang, L. Chen, Y. Bai, K. Fu, Z. Wang, and H. Xiong. Tokenselect: Efficient long-context inference and length extrapolation for llms via dynamic token-level kv cache selection. arXiv preprint arXiv:2411.02886, 2024.

C. Xiao, P. Zhang, X. Han, G. Xiao, Y. Lin, Z. Zhang, Z. Liu, and M. Sun. Infllm: Training-free long-context extrapolation for llms with an efficient context memory. In The Thirty-eighth Annual Conference on Neural Information Processing Systems, 2024a.

G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks. arXiv preprint arXiv:2309.17453, 2023.

G. Xiao, J. Tang, J. Zuo, J. Guo, S. Yang, H. Tang, Y. Fu, and S. Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads. arXiv preprint arXiv:2410.10819, 2024b.

M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, et al. Big bird: Transformers for longer sequences. Advances in neural information processing systems, 33:17283–17297, 2020.

E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman. Star: Bootstrapping reasoning with reasoning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 – December 9, 2022, 2022.

F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J. Lou, and W. Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In H. Bouamor, J. Pino, and K. Bali, editors, Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6–10, 2023, pages 2471–2484. Association for Computational Linguistics, 2023a.

K. Zhang, J. Li, G. Li, X. Shi, and Z. Jin. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In L. Ku, A. Martins, and V. Srikumar, editors, Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11–16, 2024, pages 13643–13658.

Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. Advances in Neural Information Processing Systems, 36:34661–34710, 2023b.

J. Zhao, Z. Fang, S. Li, S. Yang, and S. He. Buzz: Beehive-structured sparse kv cache with segmented heavy hitters for efficient llm inference. arXiv preprint arXiv:2410.23079, 2024.

Z. Zhou, C. Li, X. Chen, S. Wang, Y. Chao, Z. Li, H. Wang, R. An, Q. Shi, Z. Tan, et al. Llm $\times$ mapreduce: Simplified long-sequence processing using large language models. arXiv preprint arXiv:2410.09342, 2024.

A. Example of AIME Results

Prompt:

" | User | Let x, y and z be positive real numbers that satisfy the following system of equations:

$$\log_2\left(\frac{x}{yz}\right) = \frac{1}{2}$$

$$\log_2\left(\frac{y}{xz}\right) = \frac{1}{3}$$

$$\log_2\left(\frac{z}{xy}\right) = \frac{1}{4}$$

Then the value of $\left|\log_2(x^4y^3z^2)\right|$ is $\frac{m}{n}$ where m and n are relatively prime positive integers. Find $m+n$. Please reason step by step, and put your final answer within $\square$. | Assistant | "

Our Result:

⟨think⟩...⟨/think⟩ (2275 tokens)

Given the system of equations:

1. $\log_2\left(\frac{x}{yz}\right) = \frac{1}{2}$ 2. $\log_2\left(\frac{y}{xz}\right) = \frac{1}{3}$ 3. $\log_2\left(\frac{z}{xy}\right) = \frac{1}{4}$

We convert these logarithmic equations to their exponential forms:

1. $\frac{x}{yz} = \sqrt{2}$ 2. $\frac{y}{xz} = 2^{1/3}$ 3. $\frac{z}{xy} = 2^{1/4}$

Expressing each variable in terms of x, y, and z:

1. $x = \sqrt{2} \cdot yz$ 2. $y = 2^{1/3} \cdot xz$ 3. $z = 2^{1/4} \cdot xy$

Substituting these expressions into each other, we find:

- From $x = \sqrt{2} \cdot yz$ and $y = 2^{1/3} \cdot xz$, we get $x = \sqrt{2} \cdot (2^{1/3} \cdot xz) \cdot z$, leading to $z = 2^{-5/12}$. - Substituting z into the equation for y, we get $y = 2^{-3/8}$. - Substituting z and y into the equation for x, we get $x = 2^{-7/24}$.

Next, we compute $x^4y^3z^2$:

- $x^4 = (2^{-7/24})^4 = 2^{-7/6}$ - $y^3 = (2^{-3/8})^3 = 2^{-9/8}$ - $z^2 = (2^{-5/12})^2 = 2^{-5/6}$

Multiplying these together:

$$x^4y^3z^2 = 2^{-7/6} \cdot 2^{-9/8} \cdot 2^{-5/6} = 2^{-75/24} = 2^{-25/8}$$

Taking the logarithm base 2:

$$\log_2(x^4y^3z^2) = -25/8$$

The absolute value is:

$$|\log_2(x^4y^3z^2)| = 25/8$$

Thus, $m = 25$ and $n = 8$, giving $m + n = 33$.

$$\boxed{33}$$

Baseline Result:

⟨think⟩...⟨/think⟩ (9392 tokens)

Given the system of logarithmic equations:

$$\log_2\left(\frac{x}{yz}\right) = \frac{1}{2}$$
$$\log_2\left(\frac{y}{xz}\right) = \frac{1}{3}$$
$$\log_2\left(\frac{z}{xy}\right) = \frac{1}{4}$$

We convert these equations to exponential form:

$$\sqrt{2} = \frac{x}{yz} \implies x = \sqrt{2} \cdot yz$$
$$2^{\frac{1}{3}} = \frac{y}{xz} \implies y = 2^{\frac{1}{3}} \cdot xz$$
$$2^{\frac{1}{4}} = \frac{z}{xy} \implies z = 2^{\frac{1}{4}} \cdot xy$$

Substituting these into each other, we find:

1. $x = \sqrt{2} \cdot yz$ 2. $y = 2^{\frac{1}{3}} \cdot xz$ 3. $z = 2^{\frac{1}{4}} \cdot xy$

By solving these equations step-by-step, we find:

- $z = 2^{-5/12}$ - $xy = 2^{-2/3}$ - $y = 2^{-1/24}$ - $x = 2^{1/24}$

However, upon verifying the third equation, we realize the system might not have a solution. Therefore, we use matrix inversion to solve the system:

$$\begin{cases} \log_2(x) - \log_2(y) - \log_2(z) = \frac{1}{2} \\ \log_2(y) - \log_2(x) - \log_2(z) = \frac{1}{3} \\ \log_2(z) - \log_2(x) - \log_2(y) = \frac{1}{4} \end{cases}$$

Let $A = \log_2(x)$, $B = \log_2(y)$, $C = \log_2(z)$. The system becomes:

$$\begin{cases} A - B - C = \frac{1}{2} \\ B - A - C = \frac{1}{3} \\ C - A - B = \frac{1}{4} \end{cases}$$

We solve this system using Cramer's rule and matrix inversion. The determinant of the matrix is -2, and the solutions are:

$$A = \frac{61}{48}, \quad B = \frac{13}{16}, \quad C = -\frac{1}{6}$$

Thus, we have:

$$x = 2^{61/48}, \quad y = 2^{13/16}, \quad z = 2^{-1/6}$$

We need to find $\left|\log_2(x^4y^3z^2)\right|$:

$$4\log_2 x + 3\log_2 y + 2\log_2 z = 4 \cdot \frac{61}{48} + 3 \cdot \frac{13}{16} + 2 \cdot \left(-\frac{1}{6}\right)$$

Converting to a common denominator (48):

$$4 \cdot \frac{61}{48} = \frac{244}{48}, \quad 3 \cdot \frac{13}{16} = \frac{117}{48}, \quad 2 \cdot \left(-\frac{1}{6}\right) = -\frac{16}{48}$$

Summing these:

$$\frac{244}{48} + \frac{117}{48} - \frac{16}{48} = \frac{345}{48} = \frac{115}{16}$$

The fraction $\frac{115}{16}$ is in simplest form, so $m = 115$ and $n = 16$. Therefore, $m + n = 115 + 16 = 131$.

$$\boxed{131}$$

Prompt:

"|User |Real numbers x and y with $x, y > 1$ satisfy $\log_x(y^x) = \log_y(x^{4y}) = 10$. What is the value of xy?
Please reason step by step, and put your final answer within $\square$. |Assistant|"

Our Result:

⟨think⟩...⟨/think⟩ (15147 tokens)

Given the equations:

1. $\log_x(y^x) = 10$
2. $\log_y(x^{4y}) = 10$

We start by applying the logarithm power rule to both equations.

For the first equation:

$$\log_x(y^x) = x \cdot \log_x(y) = 10$$

Let $z = \log_y(x)$. Then, $\log_x(y) = \frac{1}{z}$. Substituting this into the equation, we get:

$$x \cdot \frac{1}{z} = 10 \implies x = 10z$$

For the second equation:

$$\log_y(x^{4y}) = y \cdot \log_y(x^4) = 10$$

Using the power rule, we get:

$$y \cdot 4\log_y(x) = 10 \implies 4y \log_y(x) = 10$$

Since $\log_y(x) = z$, this simplifies to:

$$4yz = 10 \implies yz = \frac{5}{2}$$

We now have two expressions:
1. $x = 10z$
2. $y = \frac{5/2}{z}$

Substituting $x = 10z$ and $y = \frac{5/2}{z}$ into the product xy, we get:

$$xy = (10z) \cdot \left(\frac{5/2}{z}\right) = 10z \cdot \frac{5}{2z} = 10 \cdot \frac{5}{2} = 25$$

Thus, the value of xy is $\boxed{25}$.

Baseline Result:

⟨think⟩...⟨/think⟩ (16223 tokens)

Given the problem where real numbers x and y with $x, y > 1$ satisfy the equations $\log_x(y^x) = 10$ and $\log_y(x^4y) = 10$, we need to find the value of xy.

First, we convert the logarithmic equations to their exponential forms using the change of base formula.

1. $\log_x(y^x) = 10$

$$x \log_x(y) = 10$$

Using the change of base formula, $\log_x(y) = \frac{\ln y}{\ln x}$. Since $\log_y x = \frac{9}{4}$, we have $\log_x y = \frac{4}{9}$. Thus,

$$x \cdot \frac{4}{9} = 10 \implies x = \frac{10 \cdot 9}{4} = \frac{90}{4} = \frac{45}{2}$$

2. $\log_y(x^4 y) = 10$

$$4 \log_y x + 1 = 10 \implies 4 \log_y x = 9 \implies \log_y x = \frac{9}{4}$$

Again, using the change of base formula, $\log_x y = \frac{4}{9}$.

Next, we express x and y in terms of each other:

$$x = \frac{45}{2} \quad \text{and} \quad y = \left(\frac{45}{2}\right)^{4/9}$$

We then compute the product xy:

$$xy = \left(\frac{45}{2}\right) \cdot \left(\frac{45}{2}\right)^{4/9} = \left(\frac{45}{2}\right)^{1+4/9} = \left(\frac{45}{2}\right)^{13/9}$$

However, upon re-evaluating and using another approach by setting $x = 2^a$ and $y = 2^b$, we find:

$$\log_x(y^x) = 10 \implies \frac{x \log_x y}{\log_x y} = 10 \implies x \cdot \frac{4}{9} = 10 \implies x = \frac{45}{2}$$

$$\log_y(x^4 y) = 10 \implies \frac{4 \log_y x + 1}{\log_y x} = 10 \implies 4 \cdot \frac{9}{4} + 1 = 10 \implies 9 + 1 = 10 \quad \text{consistent}$$

Finally, we find that the product xy simplifies to:

$$xy = \left(\frac{45}{2}\right)$$